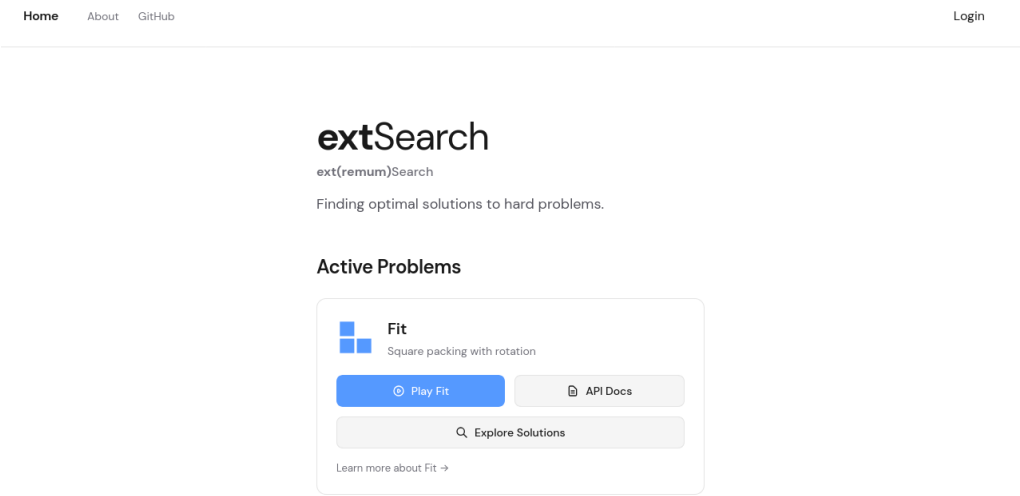
Our product ExtSearch focuses primarily on the implementation of a web-based platform which allows for human-guided dataset generation and exploration for working toward hard combinatorial optimization problems. Our system allows users to build candidate solutions for available problems hosted by the problem provider who may work with us to provide an 'index' for such problems. Additionally, this allows users to explore existing solutions to the problem through a similar viewer interface, where all data is checked automatically upon submission. We guarantee solutions are valid and provide structured storage and retrieval of results. This allows for the creation of a high-quality datasets which can be used for research and for benchmarking future algorithmic solutions both against existing data and as a testing framework via direct api calls to our server. Our customers will be research or corporate interests who have a need for human-made or human-augmented data for approaching an optimization problem, with competitors being others who aim to solve the same problem. Users will be everyday people who are interested in participating in open-source problem solving and challenges, such as for the square packing problem, with potential expertise in similar optimization problems. This is likely a limited audience, however, pools of users would likely come from math-heavy or computer science-heavy university programs with free time to use to tackle interesting problems. Our constraints are defined by our hosting environment – namely, the Kent State Computer Science Department's server which will host our landing page, authentication service, and our square-packing problem. We are limited to non-commercial domain names as public funding is used to support our hosting provider, the virtual machine provided runs Ubuntu 24.04, with Python 3.12 available, with apache2 available, as well as with the ability to setup a LetsEncrypt certificate. We have access to ports 22, 80, and 443, allowing http and https traffic and ssh for management. We handle environment security using a block-by-default approach, with 'secure' ports such as for server authentication kept internally, if additional servers are needed for customer hosting, we would allow access to a port only from a specific IP range.

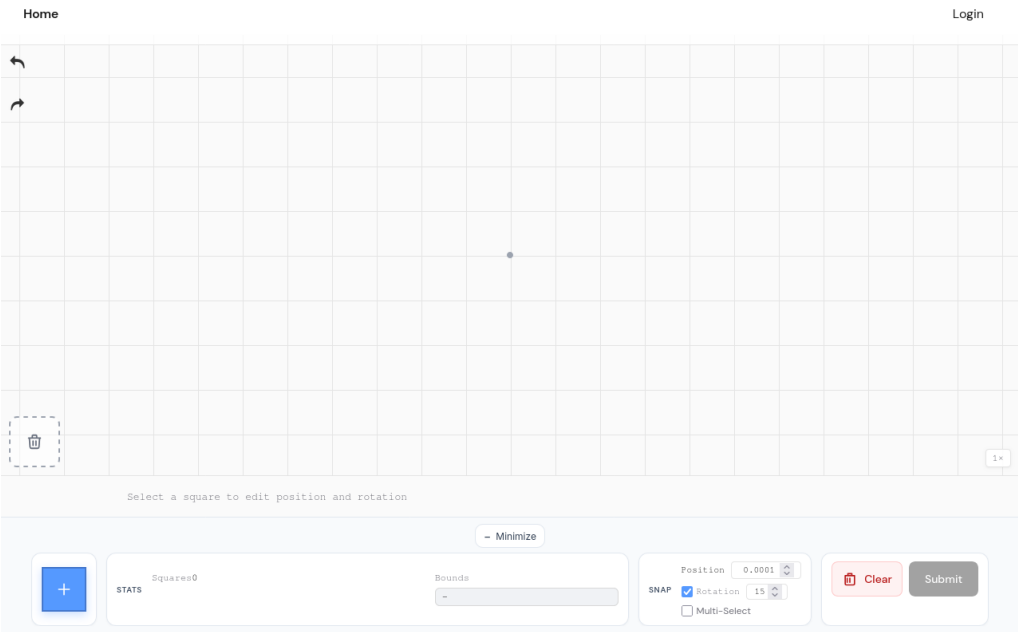
Our primary customer is Dr. Feuder Dragan who is interested in the platform as a visualization and modeling tool, with additional customers for the specific cutting stock problem. Our elicitation plan consisted of approaching Dr. Dragan and Dr. Gargi Bhaduri, and future shareholders with our existing square packing problem and structure and discussing potential constraints and limitations needed for producing a useful dataset, where we were hoping they would be able to narrow our focus to specific constraints or subproblems. We assume that individual problems will be implemented separately based on our example problem setup, and will interface similarly – hence why we provide a flexible configuration which does not enforce strict constraints on data type. We also assume that problem servers and the authentication server are able to securely communicate between each other, while server-user connections are assumed to be potentially insecure. We assume that problem-server configuration is done once, left running, then data export is handled manually after collection is completed. Finally, we also assume that API users will manually handle interfacing with the servers based on our documentation. If any of these assumptions are not held, modifications would need to be made to the frontend UI or additional package interfaces would need to be provided. Security-wise our authentication system uses public-private two-way encryption for sensitive data ontop of https, thus connection compromises should not lead to account compromise. The primary risks exist in our timeline for completion as we have taken on an ambitious project with a large scope, as well as a very large user count, as our hosting provider does not have any scaling solution.

Our UI constraints are flexible – as we expect problem servers to provide their own interface dependant on the problem being solved, however, we expect a similar interface between all sections, with a provided navigation bar on the front page and simplified bar for problem servers (including only return-to-home and login interface). Within problems, we expect the primary interface bar at the bottom of the screen,

with auxiliary interfaces/tools at the left and right sides respectively. User traversal is expected to begin at the main page, with each problem page treated as a ‘new’ site, where the user will navigate back to the index to select another problem, explore existing solutions, or read through API docs; primarily to allow for savy users to have one or more pages open at once during testing and development for API or future problems. Users are not required to authenticate to view or submit any contents (with some limitations on submission rate without an account) to facilitate ease of use and exploration for new users, with account creation available at any time, where account creation will also update submission names where possible (ie. where the authentication token is preserved with submission). Below are the primary pages, including the index and problem-server-provided pages.



(Play – primary interface for viewing and editing existing solutions)



(API Docs – information needed to interface with the server endpoints)

Home About GitHub Login

Fit API

Programmatic access to submit square packing solutions from any language.

Quick Start

Two steps to submit solutions from your own code:

1. **Get a token**: exchange your username & password for a JWT.

```
curl -X POST https://extsearch.org/api/fit/token \
    -H "Content-Type: application/json" \
    -d '{ "username": "you", "password": "yourpass" }'

# → { "token": "eyJhbGciOi..." }
```

2. **Submit a solution**: send your squares with the token.

```
curl -X POST https://extsearch.org/api/fit/submit \
    -H "Content-Type: application/json" \
    -H "Authorization: Bearer YOUR_TOKEN" \
    -d '{
        "squares": [
```

(Solution Explorer includes possible sub-problems (ie. square counts) and proven ones alongside a leaderboard for)

Home About GitHub Login

Explore Solutions

Best submissions for the Fit problem by number of squares.

KNOWN OPTIMAL (PROVED · TRIVIAL)

1 2 3 4 5 6 7 8 9 10 13 14 15 16 22 23 24 25 33 34 35 36

BEST FOUND (NOT PROVED)

11 8 12 0 17 0 18 0 19 0 20 0 21 0 26 0 27 0 28 0 29 0 30 0 31 0 32 0 37 0 38 0

7 submissions with 11 squares

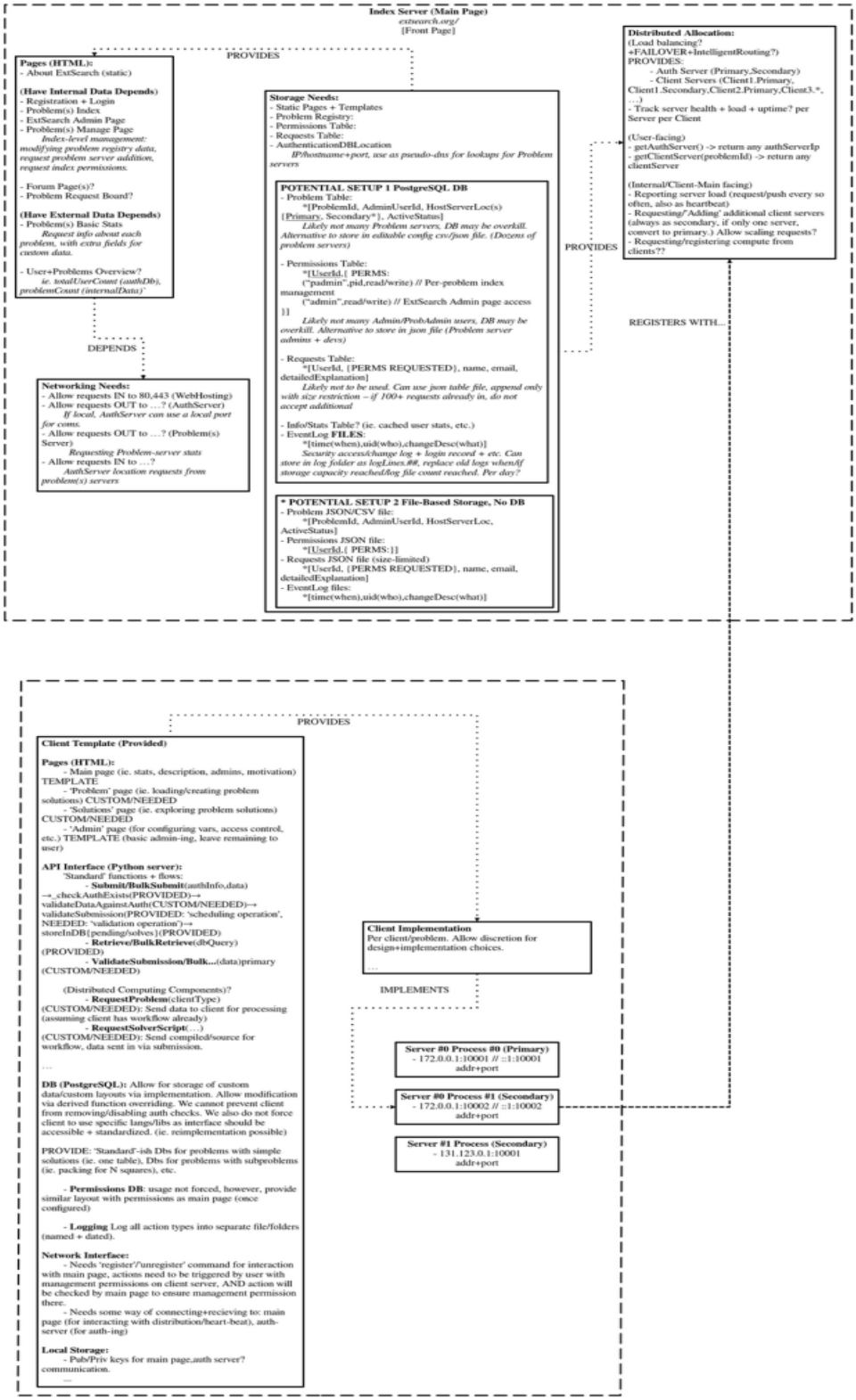
☑ Hide duplicates

#	USER	S	DATE
1	Anonymous	4 units	2026-04-07 21:07
2	Anonymous	4.0476 units	2026-04-14 19:55
3	Anonymous	4.18317 units	2026-04-16 15:42
4	Anonymous	5.3988 units	2026-04-07 21:07
5	chasekosborne	9.094 units	2026-03-31 22:14
6	test1	10.2857 units	2026-04-02 19:47
7	chasekosborne	10.7967 units	2026-03-31 22:13

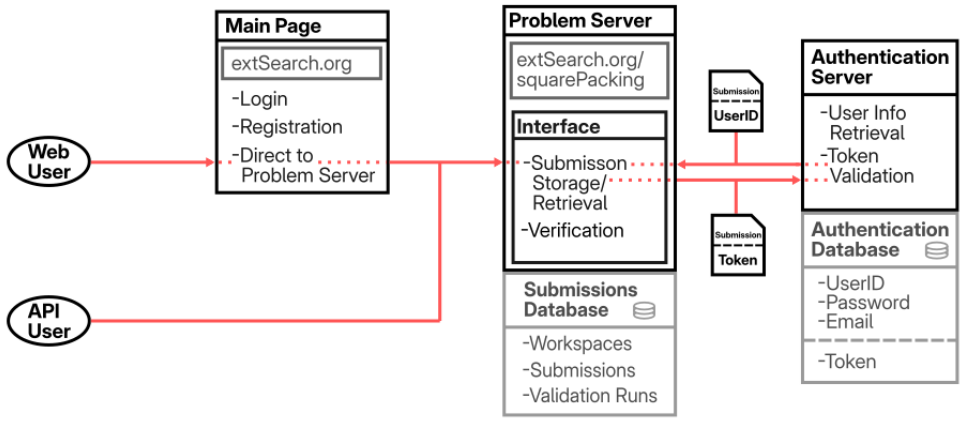
* Duplicate: another submission with the same bounding box size exists.

Most users are expected to interface through the front-page and API, while customers are expected to work through our team to deploy and connect additional problems for quality control.

Our system is setup to be distributed, with each problem server, and the authentication server instance(s) stored in lists on the index server, allowing for a distributed setup in the future.

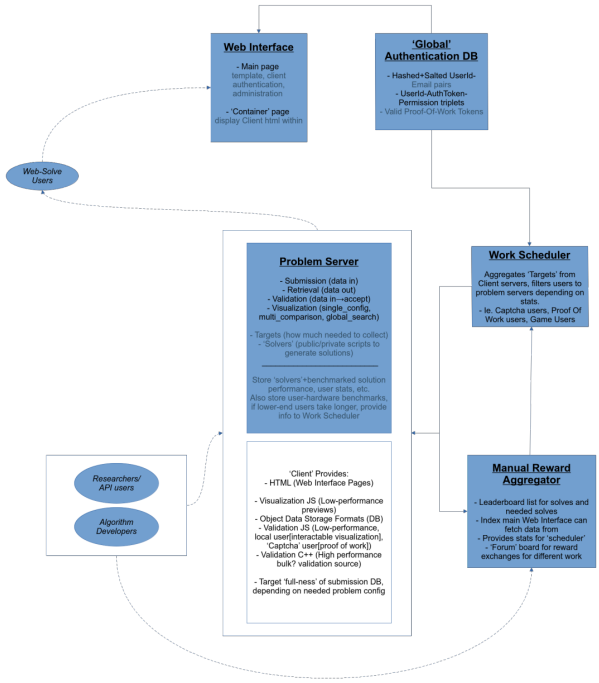


the above image provides our primary interface between the index and problem servers, as well as the backend interface details.

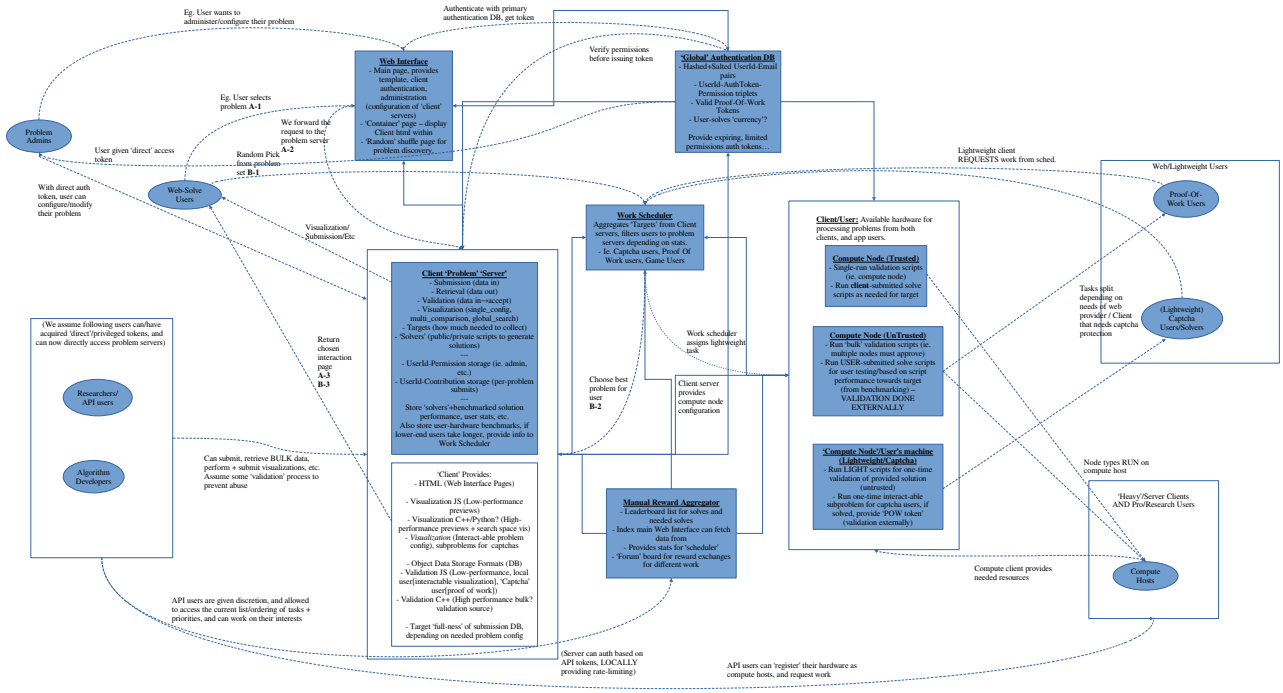


(User authentication flows)

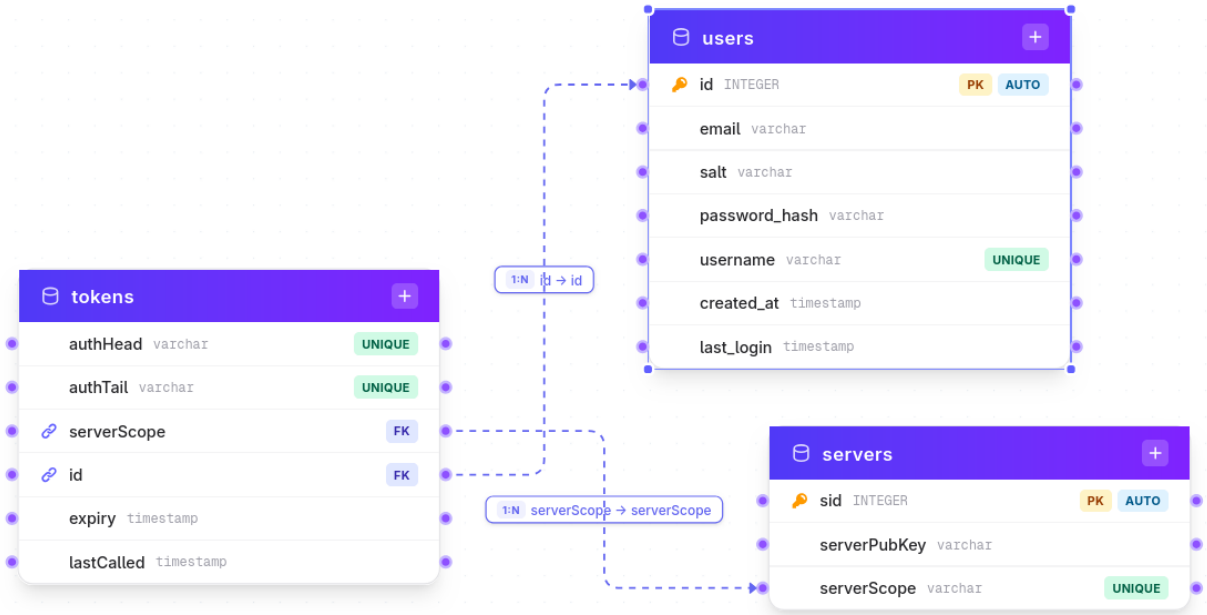
With available functions within the problem server, we are also able to provide a 'scheduler' for validating solutions as follows:



API users are able to bypass the front-page and use provided authentication tokens scoped to individual problem servers, while UI-based users use the standard workflow.

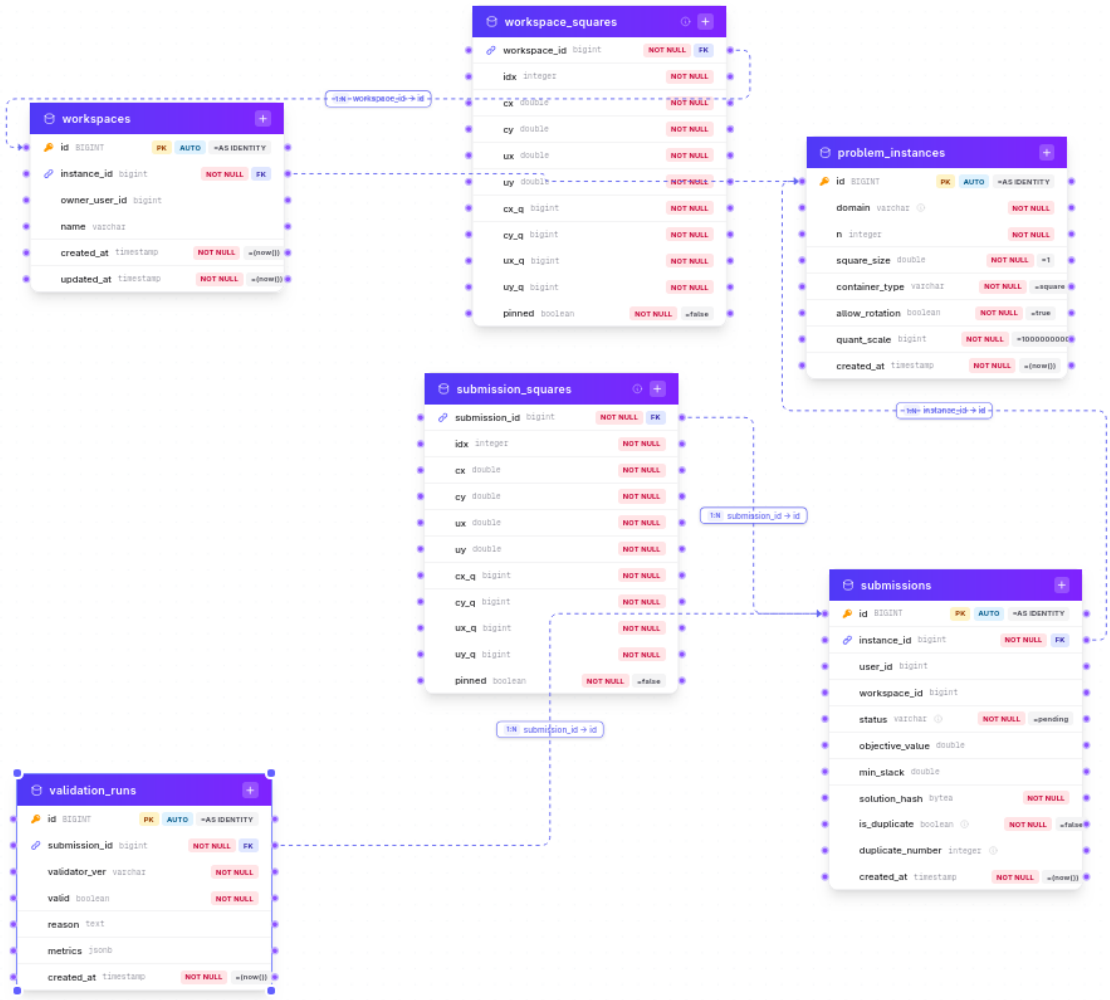


Our primary authentication schema contains 3 tables: (excluding server routing)



Where users are able to hold access tokens (after authentication) scoped to servers (including index). Each user action must be 'signed' with the full token, and include a 'challenge' computed from a hash of a shared key combined with a timestamp and the full key, actions are only considered authenticated if the server-side computation matches the users. Tokens allow for time-independent access as long as each access is sequential, with multiple applications needing multiple access tokens even when acting on a single server, acting as a unique key per-application.

Problem server data is stored as follows:



Where each submission has a unique ID, and may not be validated in real-time, for example, under heavy system loads or with a distributed computing setup with a queue before spawning a worker. The submissions tracks raw data stored locally in containers, allowing for the DB to handle a large number of submissions while object data storage is independent. This also allows for DB-independent data exporting, and for adaptation to different data types by customers without modifying the underlying storage database, data format versioning is expected to be handled by the customer.

Server lists are stored on the index server, thus when validating data (both for data integrity and authentication) the server contacts the index server to fetch an authentication instance to validate a challenge+result pair and receive the authTail part of the key for authentication. As keys are scoped, a server compromise only leaks the authentication token needed for data submission on that server, and does not allow for malicious submissions or account data modification.

We keep the authentication and submission sides of the system isolated to allow for flexibility and security, where customers and we do not have to worry about their server's security, as a compromise would be limited to their system and submitted data and not compromise the data of our users.

Performance-wise we had a low target (under 100 concurrent online), and thus impact is minimal. We use a non-blocking submission and validation flow for data input, thus the users would not be affected by server slowdowns. Throughput is limited primarily by network connection and available processing power on the targeted problem server for validation. Our index server (primary) will be always available, and has little processing requirements, thus responses will be fast, and the main pages will be loaded quickly and always available. The authentication server also has limited processing needs, with most actions being non-blocking. However, token generation may lead to some slowdown due to the random head and tail generation, thus it has been limited to ten thousand attempts before failing. Problem servers will have the primary load in our system, and will be dependent on the hardware running them, ideally, response time will be fast, however, on increased load, a longer delay is acceptable. We account for this with an allowed time-delayed authentication scheme for data submissions.

For security, we have isolated our primary user data to one server, with limited external access. Passwords are hashed with salt, and only transmitted during user signup. Full-length authentication tokens are also only transmitted during the requests and not afterwards. Additionally, both tokens and user-logins are time-restricted to avoid replay attacks, and tokens allow for the signing and verification of data without the transmission of a full access token, while also functioning without sending large data files to the authentication server.

By making data validation and processing a separate process, we also are able to achieve a higher reliability within the main servers as any potential malicious data would be processed externally, and only stored by the main server. Thus, even if the validation process is to crash, the main server stays up allowing submissions, while the validation process can be restarted as needed. Further, the authentication server has no capacity to crash, as all code is written within exception-catch blocks, where a method may fail silently but not take down the process.

Due to time constraints we were not able to develop a management portal through which a user could start and stop individual processes or problem servers, thus for updating problem server code and exporting collected data, an administrator would need to remotely connect to the machine, download or upload needed data, and then restart the server processes. This restart can also be scheduled using a cron job to have predictable downtime for updates. This is an acceptable compromise as little active server administration is needed after configuration and before data export, where the expected customer will leave the server running until data is needed.

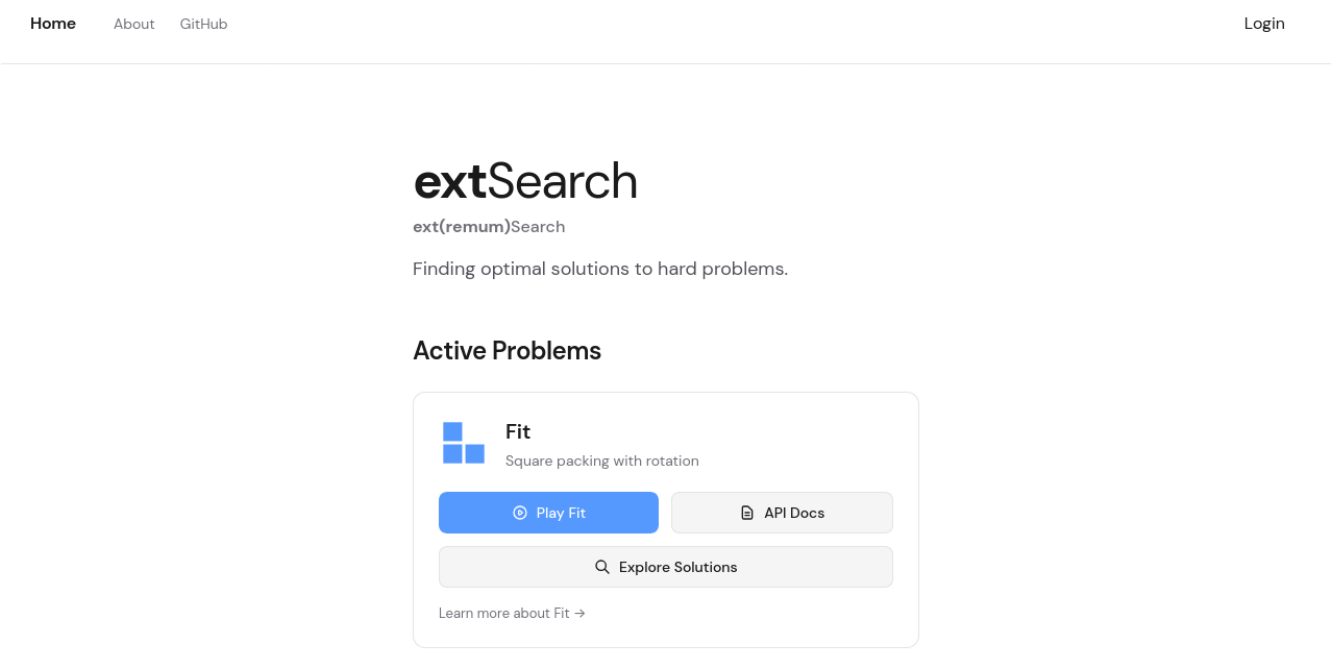
There are multiple places where we expect errors to occur: namely, within user-generated submission data, api interfaces, and data validation processes. The primary system state issues that can occur is during initial setup, where all processes must be running for the server to function. If the database backend is not available, logins, signups, and submitted data will be unavailable, this is noted in the logs when starting up the processes, and can be addressed by starting the needed server. Once all processes are running, the system is considered stable as no external user action should cause any of the processes to become unavailable. For redundancy, each process can be launched in a loop, where on crash, the process will restart, discarding the offending data and continuing from the last saved stable state and data set. During the recovery procedure, the server and data access may become unavailable to users, however,

once the process is restored, user connections will resume and should process the backlog of connections. The system state will always fallback to the most recent saved data commit on crash and restore, with the only risk coming from server hardware failures, which we will leave to the customer as they are responsible for their data hosting.

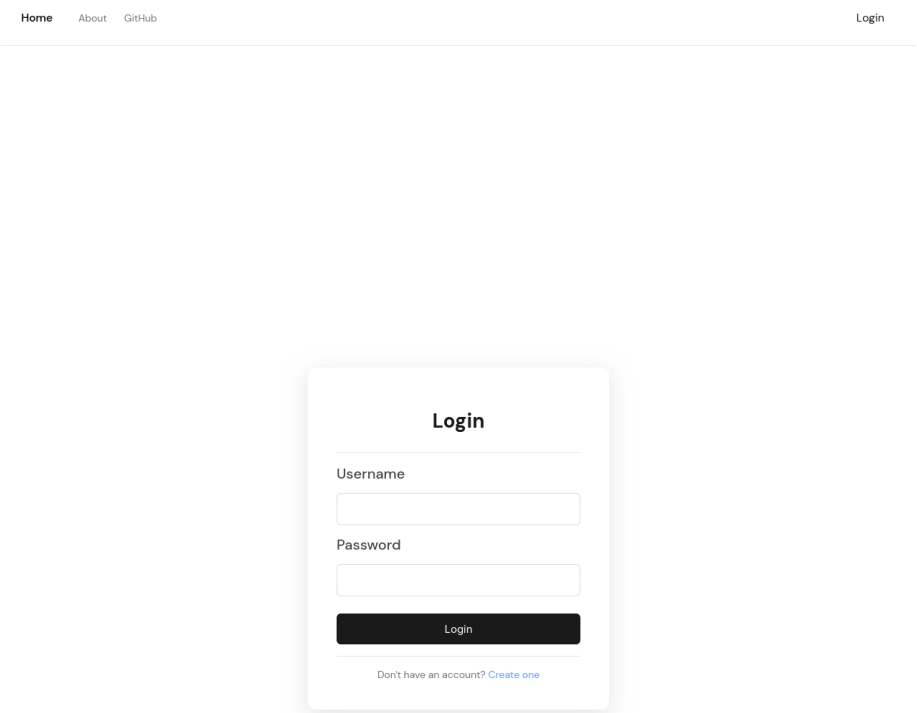
In the future, given more time, our goal would be to complete the full isolation of the example problem server and the index server, setting up routing endpoints using nginx, and completing the template problem server to make development for future problem servers plug-and-play for customers with existing tools. We also would be able to finalize example problems servers with different applications such as the cutting stock and fabric cutting problems with optional constraints for solutions.

Thank you to everyone on the team for their work during development, the Kent State CS Department for providing hosting, and Dr. Feuder Dragan for inspiring and evaluating our work!

User guide: extsearch.org



1) Login



2) Create an account

Create Account

Username

test1111

Email

email@email.com

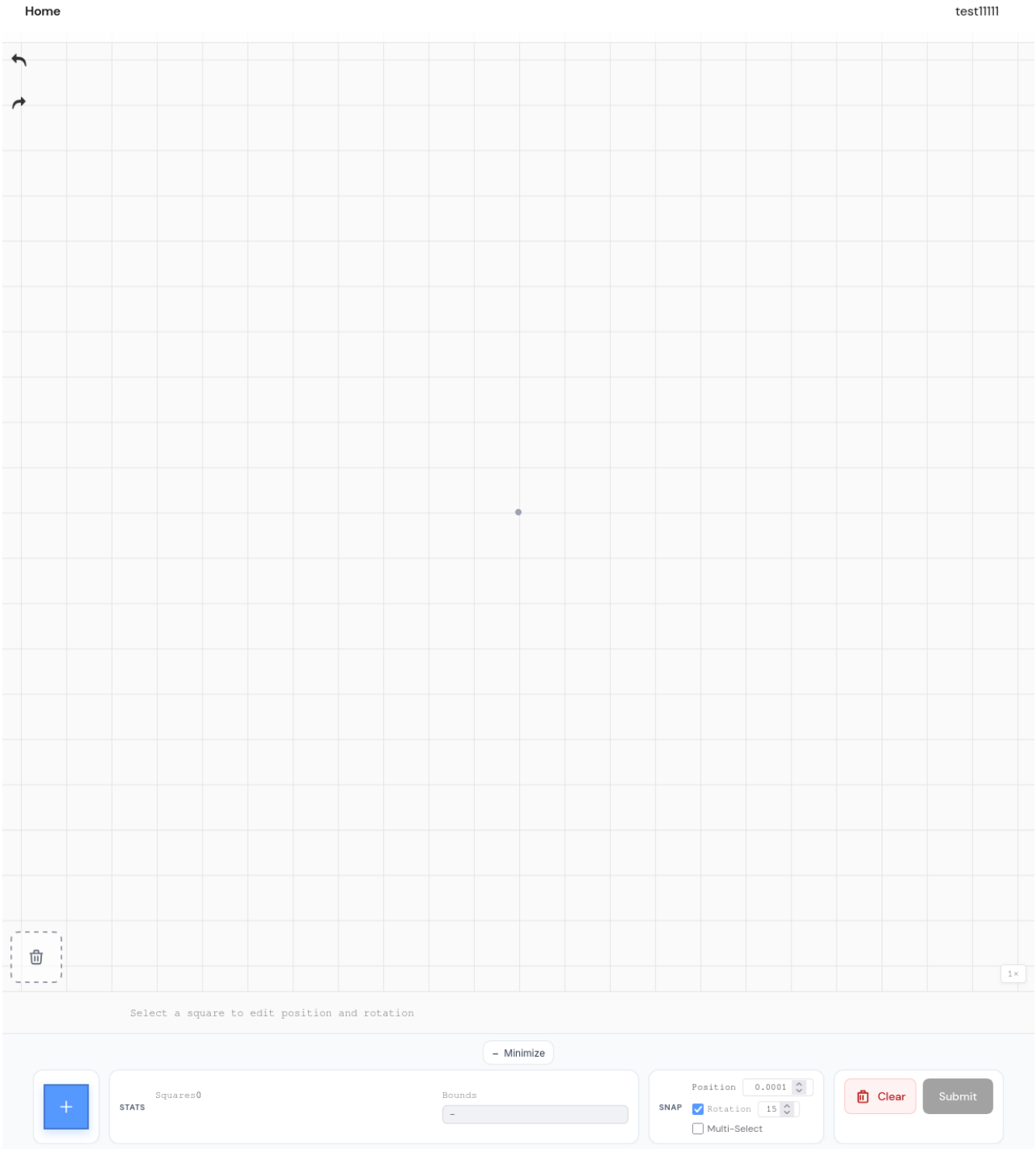
Password

●●●●●●●●

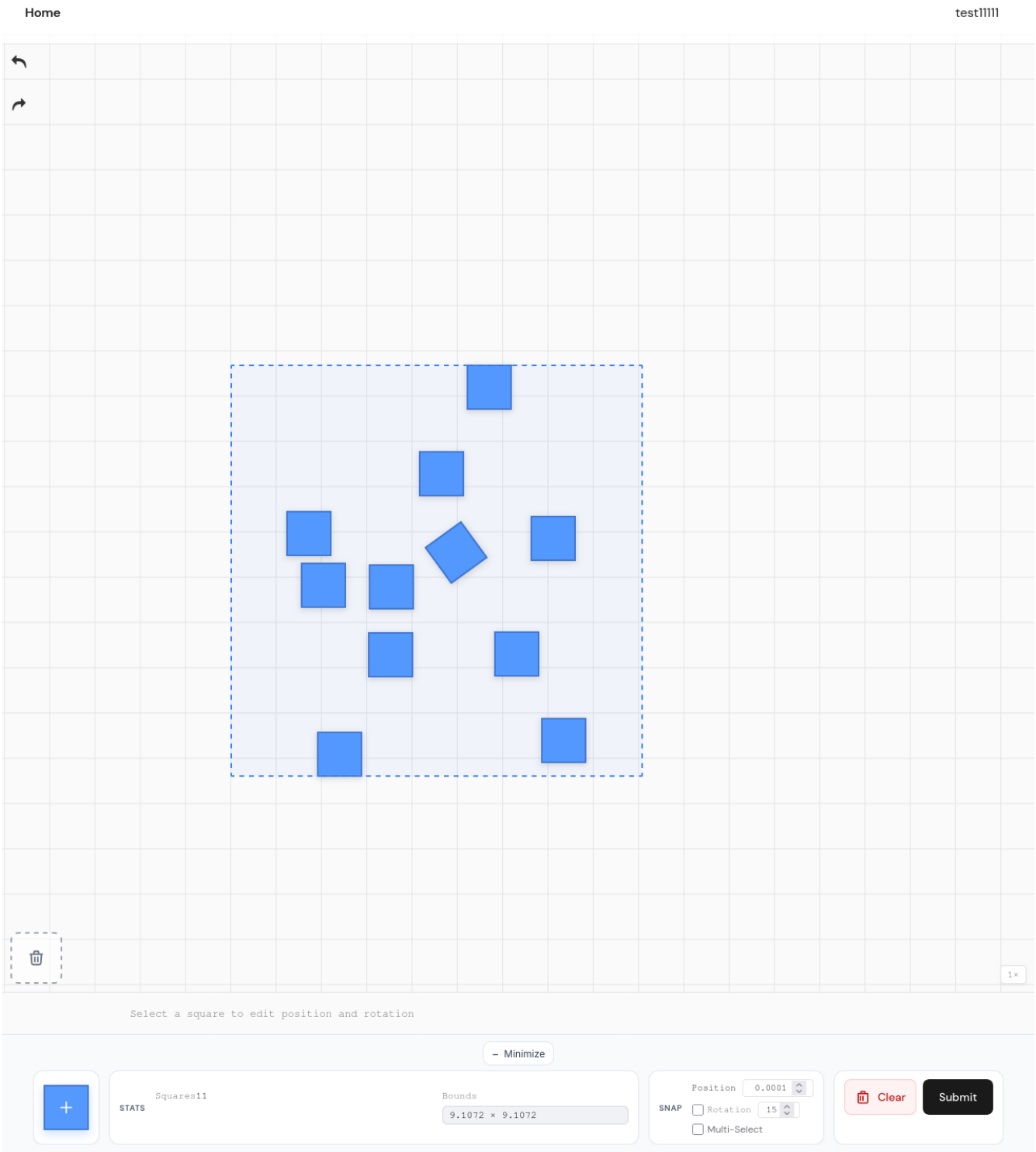
Create Account

Already have an account? Log in

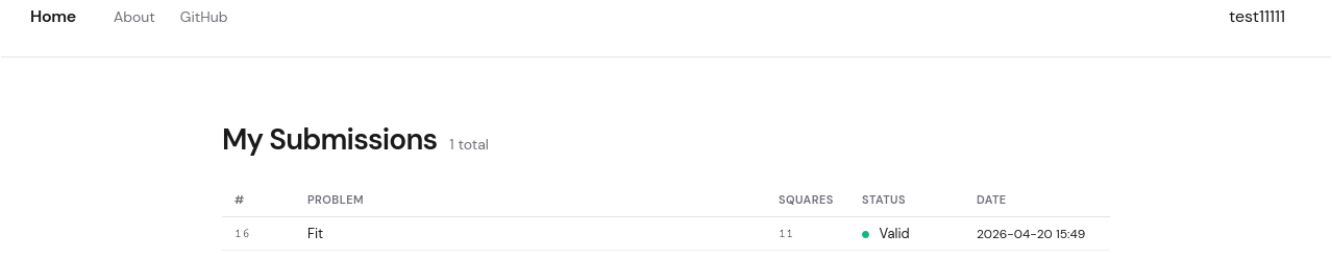
3) Play Fit!



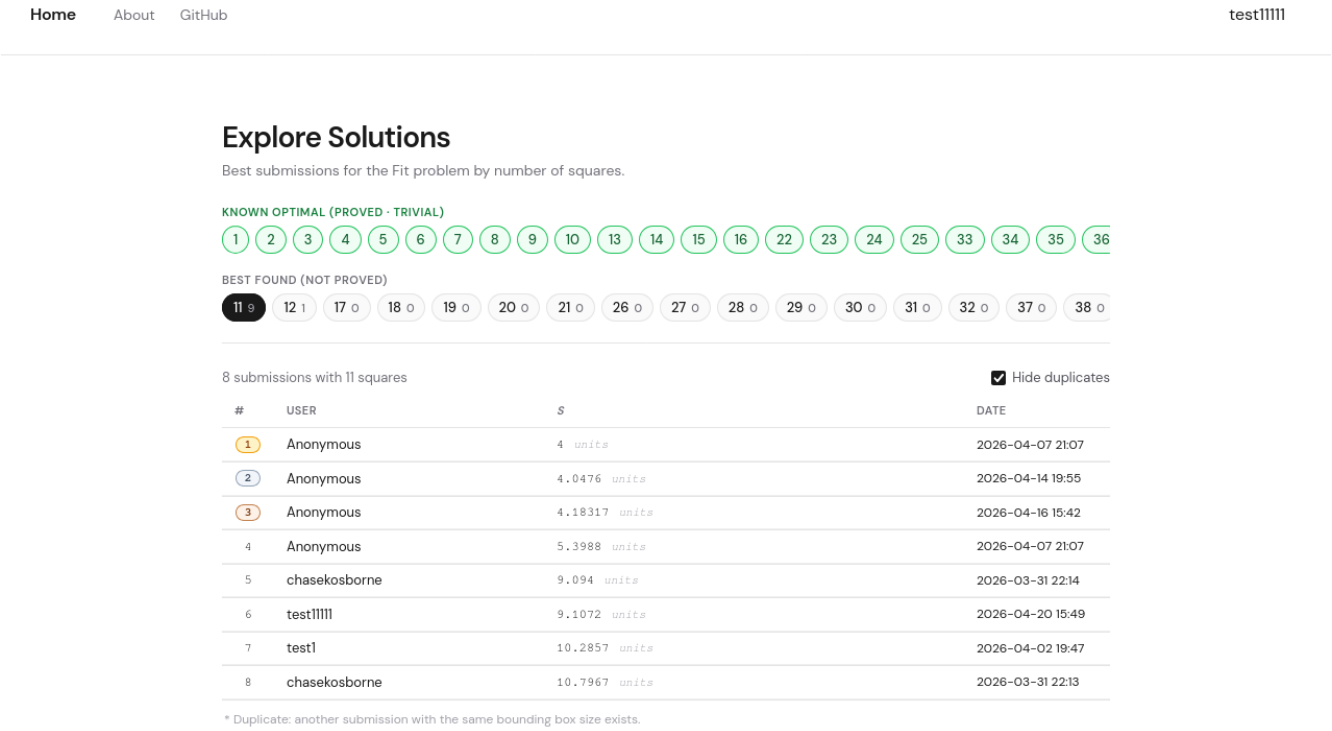
4) Click+drag squares out to move them into place. Double click to switch between rotation and movement, submit solution once all issues are resolved and submit button is available.



5) View your submissions: (under your account)



6) View game submissions: (home→explore solutions)



7) Bulk contribute via API:

Home About GitHub test11111

Fit API

Programmatic access to submit square packing solutions from any language.

Quick Start

Two steps to submit solutions from your own code:

1. **Get a token**: exchange your username & password for a JWT.

```
curl -X POST https://extsearch.org/api/fit/token \
    -H "Content-Type: application/json" \
    -d '{ "username": "you", "password": "yourpass" }'

# → { "token": "eyJhbGciOi..." }
```

2. **Submit a solution**: send your squares with the token.

```
curl -X POST https://extsearch.org/api/fit/submit \
    -H "Content-Type: application/json" \
    -H "Authorization: Bearer YOUR_TOKEN" \
    -d '{
          "squares": [
              [{"x": 0, "y": 0}, {"x": 56, "y": 0}, {"x": 56, "y": 56}
              ...
          ]
        }'
```

Authentication

8) Check out other active problems on the homepage, follow similar steps...

extSearch

ext(remum)Search

Finding optimal solutions to hard problems.

Active Problems

SQUARE

Fit

Square packing with rotation.

Play Square | API Docs

Explore Solutions

Learn more about Fit →

RECTANGLE

Fit

Rectangle packing with rotation.

Play Rectangle | API Docs

Explore Solutions

Learn more about Fit →